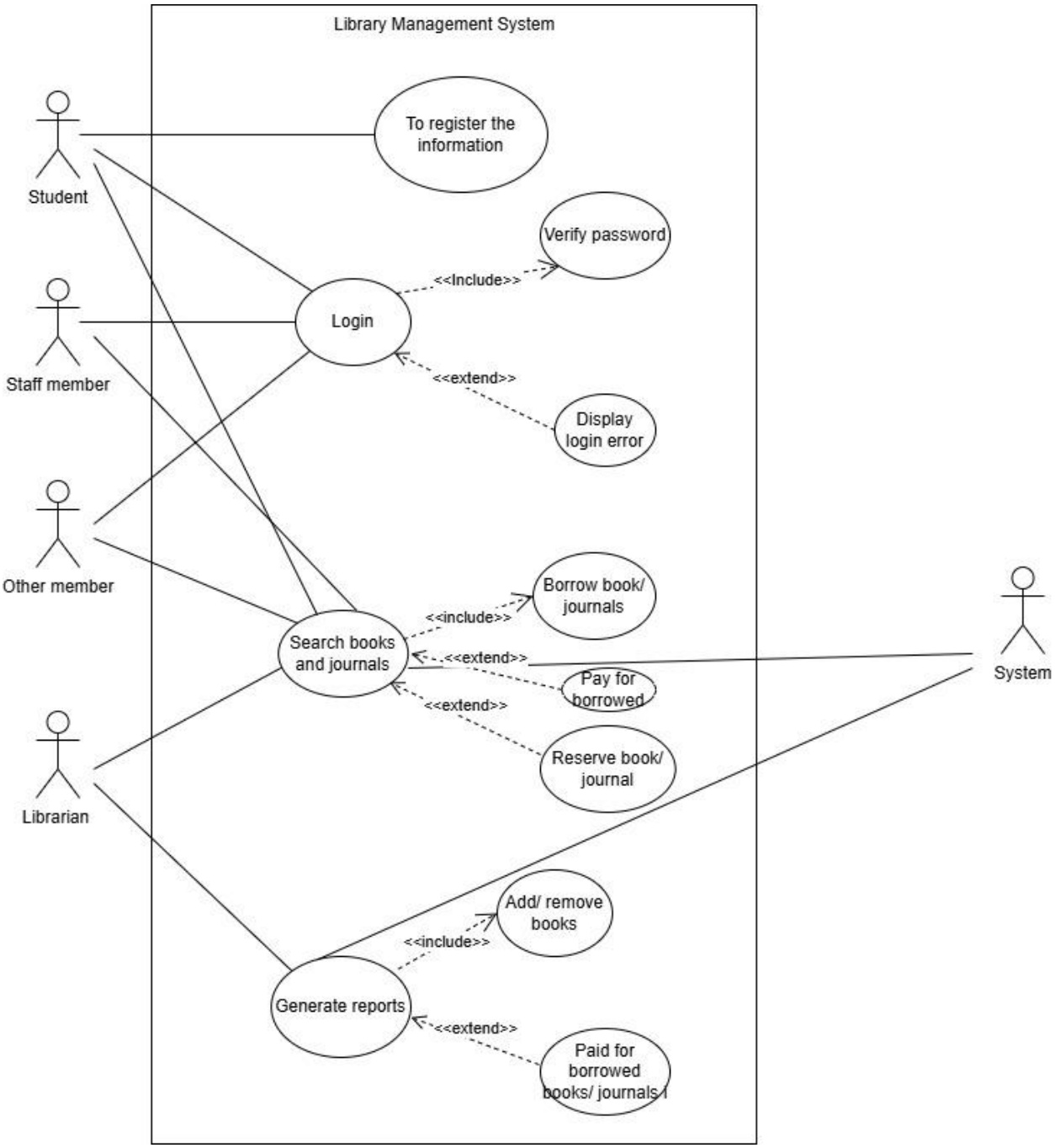
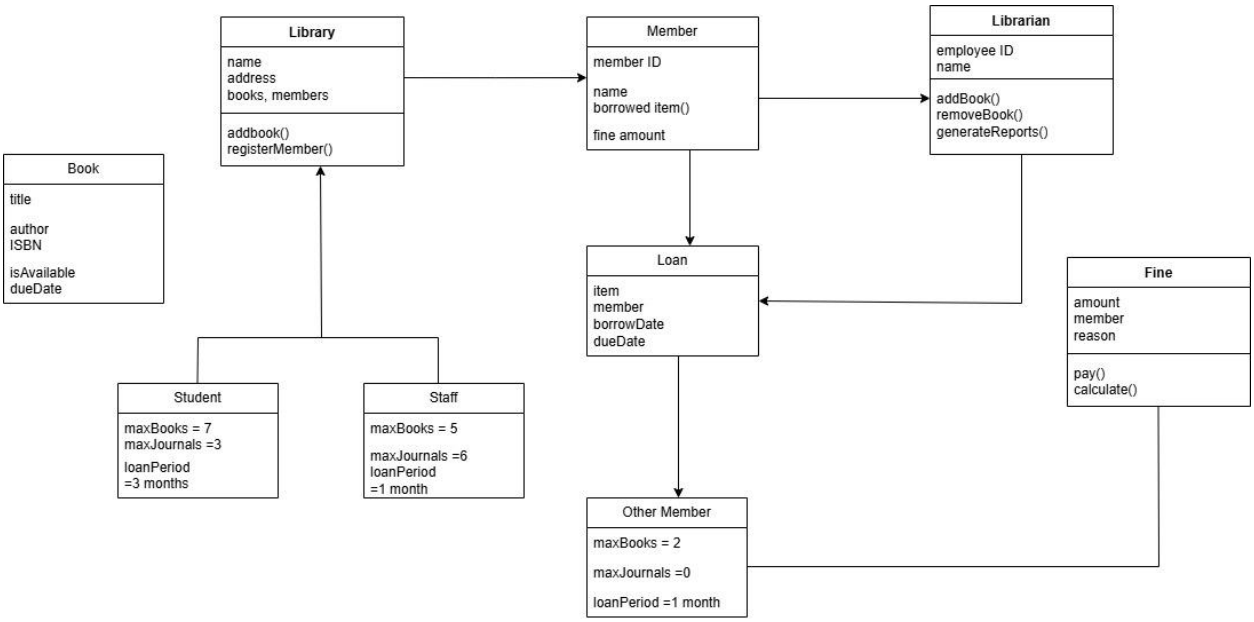
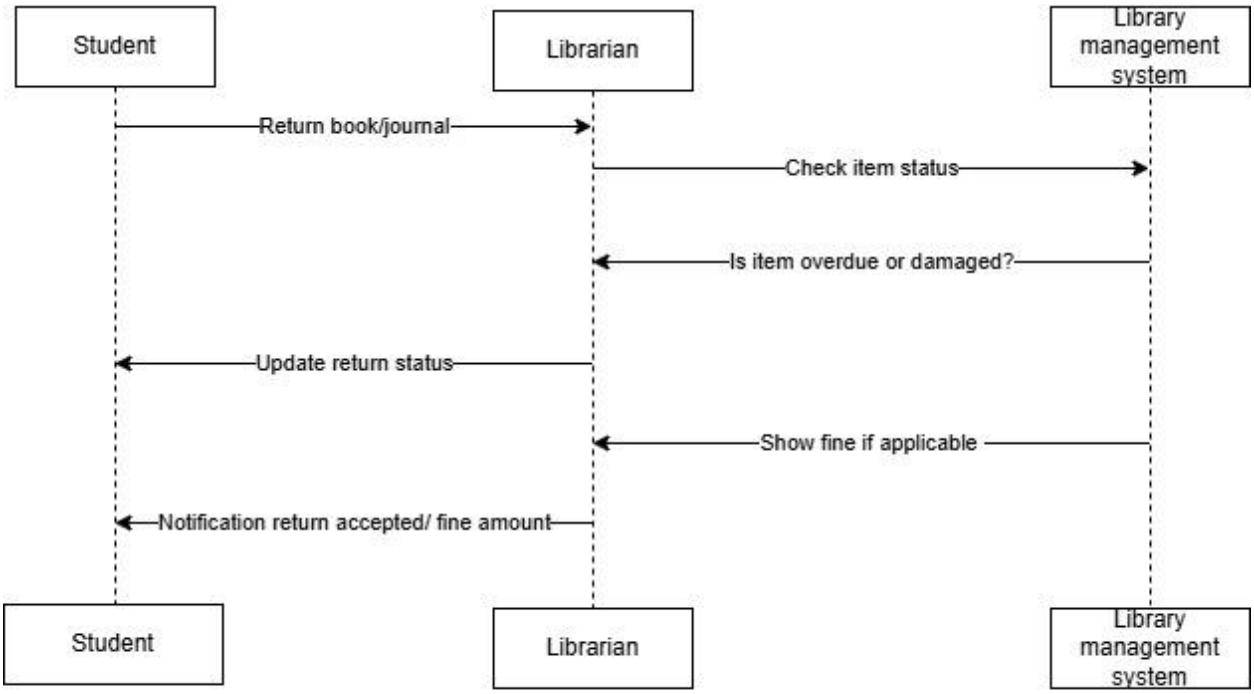

Library Management System
Student
Staff member
Other member
Librarian
System
To register the information
Verify password
<<Include>>
Login
<<extend>>
Display login error
Borrow book/ journals
<<include>>
Search books and journals
<<extend>>
Pay for borrowed
<<extend>>
Reserve book/ journal
Add/ remove books
<<include>>
Generate reports
<<extend>>
Paid for borrowed books/ journals

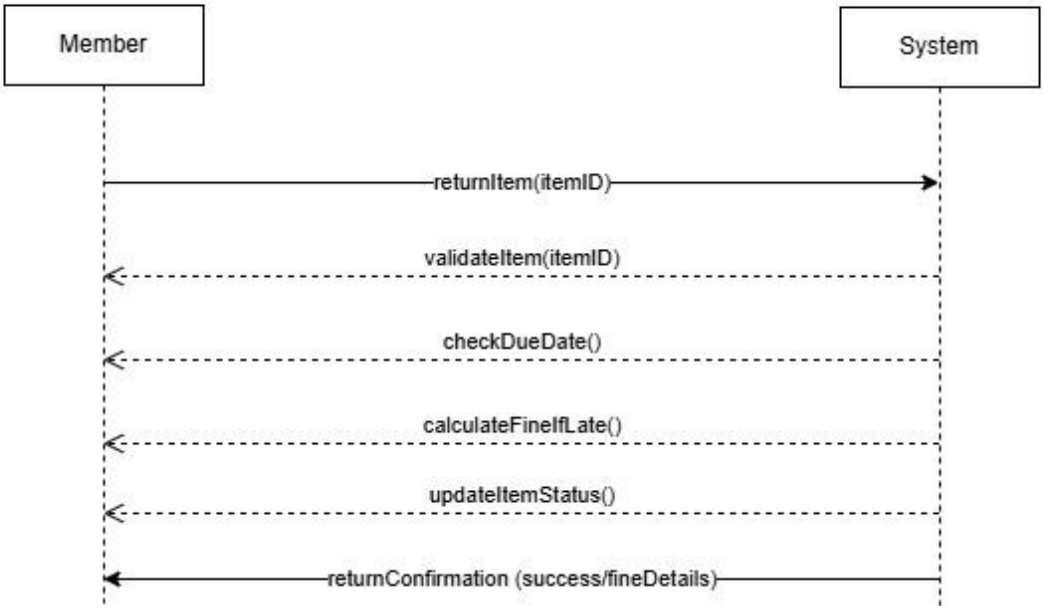
Use Case :	Borrowing books/journals
Goal in context:	This use case allows a student and other members, who is not a customer of library to borrow books/ journals if his/ her daily borrowing limit allows it
Preconditions:	The person must already have a library account. They must be logged into the system. They haven't borrowed too many items yet. The item they want is currently available.
Success End Condition:	The book or journal is issued to the person. The system records that it was borrowed. A return due date is set. The person can now see the borrowed item in their account
Failed End Condition:	The person tries to borrow more than allowed, so the system stops the request. The book or journal they want is not available (already borrowed). They're not logged in, so they can't borrow anything. Their account is blocked or has unpaid fines.
Primary Actor:	Student, Staff Member or Other Member and librarian
Secondary Actor:	System, Catalog Database.
Trigger:	The member selects a book or journal and clicks the "Borrow" button.
Description:	It lets users register as members, search for books, borrow and return them without hassle.
Extensions or variations:	if a book is already borrowed, the user can place a reservation (extension of borrowing).

Use Case :	Return books/journals
Goal in context:	The member brings back the borrowed book or journal to the library so it can be available for others to use.
Preconditions:	Member has borrowed the item. Member is logged into the system
Success End Condition:	The book or journal is marked as returned. It becomes available for others to borrow. If it was returned late, a fine is calculated and added. The person's borrowing record is updated
Failed End Condition:	The system can't find the borrowed record (maybe wrong book ID or already returned). The person isn't logged in, so the system won't accept the return. The return is too late, and the system blocks further actions until a fine is paid.
Primary Actor:	librarian
Secondary Actor:	Library System, Fine Calculator (if return is late), Loan Record System
Trigger:	The member brings back a borrowed item and chooses the "Return" option in the system (or hands it to the librarian).
Description:	If the item is returned late or damaged, the system may apply a fine.
Extensions or variations:	If the book is damaged, the user may be charged for repair or replacement

Class	Attributes	Methods (Functions)
Library	name, address, books, members	addBook(), removeBook(), registerMember()
Book	title, author, ISBN, isAvailable, dueDate	checkAvailability(), markAsBorrowed(), markAsReturned()
Journal	title, volume, issueNumber, isAvailable, dueDate	checkAvailability(), markAsBorrowed(), markAsReturned()
Member	memberID, name, type, borrowedItems, fineAmount	borrowItem(), returnItem(), payFine()
Student (inherits Member)	maxBooks=6, maxJournals=3, loanPeriod=3 months	—
Staff (inherits Member)	maxBooks=10, maxJournals=6, loanPeriod=3 months	—
OtherMember (inherits Member)	maxBooks=2, maxJournals=0, loanPeriod=1 month	—
Librarian	employeeID, name	addBook(), removeBook(), generateReports()
Loan	item, member, borrowDate, dueDate, returnDate	calculateFine(), isOverdue()
Fine	amount, member, reason	pay(), calculate()


Library
name
address
books, members
addbook()
registerMember()
Member
member ID
name
borrowed item()
fine amount
Librarian
employee ID
name
addBook()
removeBook()
generateReports()
Book
title
author
ISBN
isAvailable
dueDate
Loan
item
member
borrowDate
dueDate
Fine
amount
member
reason
pay()
calculate()
Student
maxBooks = 7
maxJournals =3
loanPeriod
=3 months
Staff
maxBooks = 5
maxJournals =6
loanPeriod
=1 month
Other Member
maxBooks = 2
maxJournals =0
loanPeriod =1 month


Student
Librarian
Library management system
Return book/journal
Check item status
Is item overdue or damaged?
Update return status
Show fine if applicable
Notification return accepted/ fine amount
Student
Librarian
Library management system


Member
System
returnItem(itemID)
validateItem(itemID)
checkDueDate()
calculateFineIfLate()
updateItemStatus()
returnConfirmation (success/fineDetails)

GRASP Principles in My Library Management System

1. Information Expert

I gave jobs to the classes that actually know the data.
For example, the **Member** class knows how many books someone has, so it decides if borrowing is allowed. Same with the **Book** — it knows if it's available or not.

2. Creator

The class that needs something should be the one to make it.
When a member borrows a book, a **Loan** record is needed. Since the member is doing the borrowing, it makes sense for the **Member** class to create the **Loan**.

3. Controller

One class should handle the main actions in the system.
In this case, the **LibrarySystem** (or maybe a **Librarian**) handles requests like borrowing and returning. It talks to other parts and makes sure things happen in the right order.

4. Low Coupling

Keep classes from depending too much on each other.
For example, **Fine** just deals with fines. **Book** deals with its own data. They don't need to know much about each other. This makes updates or changes easier later.

5. High Cohesion

Each class should stick to one main job.
Like the **Loan** class — it handles due dates, returns, and late fees. All related stuff is kept in one place.

6. Polymorphism

Let similar classes act differently if needed.
All members borrow items, but **students**, **staff**, and **others** have different borrowing limits. So I gave each type its own rules by using subclasses.

7. Pure Fabrication

Sometimes, you need to create a helper class just to keep things neat.
I made a **ReportGenerator** to create reports. It's not a real thing in a library, but it helps separate that logic from everything else.

8. Indirection

Use a go-between to manage communication.
Instead of letting a **Member** talk to a **Book** directly, I made the **LibrarySystem** handle that. It keeps everything more organized and easier to control.

9. Protected Variations

Design the system in a way that changes won't break everything.
If we want to add eBooks or change how fines work later, we won't need to redo the whole system — because each part is kept flexible.

B. The tools you used to accomplish these tasks (and attach a link, is possible, to access your work).

To complete the tasks in this library management system project, I used a tool that helped me create the diagrams and organize my ideas. Here's a simple breakdown which is:

Draw.io (also known as diagrams.net)

I used this tool to draw all the diagrams like the use case, class diagram, and sequence diagram. It’s really user-friendly and works straight from the browser without needing to install anything. It also lets you save your work to Google Drive or your computer. And this is my link of my work

A. the tools you used to accomplish these tasks (and attach a link, is possible, to access your work).